

AI Transition Master Roadmap

From Software Engineer → AI Engineer → Agentic AI Systems Engineer

Author: Ranjan Mondal

Focus Areas: RAG, Agentic AI, Multi-Agent Systems, LLMOps, Production AI Systems

1. Vision

This roadmap documents the complete AI career transition journey from traditional software engineering into production-grade AI systems engineering.

The goal is not just learning AI concepts, but becoming capable of:

- Building production AI systems
- Designing RAG architectures
- Building AI agents
- Designing multi-agent systems
- Applying evaluation and guardrails
- Understanding deployment and scalability
- Building portfolio-grade AI products

The transition is heavily aligned with:

- Modern AI industry hiring trends
 - Agentic AI systems
 - Retrieval-Augmented Generation (RAG)
 - LLMOps / MLOps
 - Real-world system design
-

2. Current Background

Existing Strengths

Already strong in:

- Production-grade software engineering
- Backend systems
- MERN stack

- Python stack
- Java ecosystem
- APIs
- Scalable application architecture
- Real-world debugging and deployment
- System thinking

This is a major advantage because:

Most AI learners know models. Few know systems.

The goal is to combine:

Software Engineering + AI = AI Systems Engineer

3. Career Transition Objective

Primary Goal

Transition into roles such as:

- AI Engineer
 - LLM Engineer
 - Applied AI Engineer
 - GenAI Engineer
 - AI Platform Engineer
 - Agentic AI Engineer
-

4. Core Learning Philosophy

The transition is based on this principle:

Build while learning.

Not:

Learn everything first → Build later.

Instead:

Learn concept → Apply immediately → Improve through iteration.

This prevents:

- Tutorial paralysis
 - Endless revision loops
 - Over-learning without proof
 - Burnout
-

5. Long-Term Technical Direction

The roadmap progresses through the following layers:

Phase 1 — RAG Systems

Learn:

- Embeddings
- Chunking
- Retrieval
- Vector databases
- Prompt engineering
- Context management

Outcome:

Build reliable grounded AI systems.

Phase 2 — Agentic AI

Learn:

- Tool calling
- Multi-step reasoning
- Task planning
- Memory systems
- Workflow automation

Outcome:

Build AI systems that perform tasks, not just answer questions.

Phase 3 — Multi-Agent Systems

Learn:

- Agent orchestration
- Role-based agents
- Collaboration patterns
- Evaluation agents
- Shared memory

Outcome:

Build coordinated AI systems.

Phase 4 — LLMOps / Productionization

Learn:

- Evaluation
- Monitoring
- Logging
- Guardrails
- Cost optimization
- Latency optimization
- Deployment

Outcome:

Production-grade AI engineering.

6. Complete Project Roadmap

A total of 3 major AI projects are planned.

These projects are intentionally progressive.

Each project builds on the previous one.

PROJECT 1 — Production-Grade RAG System

Title

RAG-Based Knowledge Assistant

Objective

Build a Retrieval-Augmented Generation (RAG) system capable of:

- Processing PDFs
 - Understanding invoice/document content
 - Performing semantic search
 - Generating grounded answers
-

Key Concepts Covered

- PDF ingestion
 - Chunking
 - Embeddings
 - FAISS vector database
 - Semantic retrieval
 - Prompt engineering
 - Context expansion
 - Retrieval optimization
 - FastAPI
 - Streamlit UI
 - Open-source LLM integration
-

Technologies Used

Core

- Python
- FastAPI
- Streamlit
- LangChain
- FAISS
- Sentence Transformers

LLMs

- TinyLlama
- Groq API
- Llama 3.x

Embeddings

- sentence-transformers/all-MiniLM-L6-v2
-

Important Engineering Decisions

Neighbor-Aware Retrieval

Problem:

Chunk fragmentation caused incomplete retrieval.

Solution:

Retrieve adjacent chunks to preserve semantic continuity.

Startup Optimization

Problem:

Embeddings and models loaded repeatedly.

Solution:

Use FastAPI startup lifecycle to preload:

- PDFs
 - embeddings
 - vector store
 - LLM
-

Latency Optimization

Problem:

TinyLlama inference too slow on local hardware.

Solution:

Switch to Groq-hosted inference.

Final Architecture

User Query ↓ FAISS Retrieval ↓ Relevant Chunks ↓ Neighbor Expansion ↓ LLM Generation ↓ Final Grounded Response

Planned Production Features

Completed

- PDF ingestion
 - Chunking
 - Embeddings
 - Vector search
 - Open-source RAG
 - FastAPI backend
 - Streamlit UI
 - Startup optimization
 - Neighbor-aware retrieval
-

Pending

- Evaluation framework
 - Logging system
 - Better retrieval metrics
 - Hybrid search
 - Metadata filtering
 - Deployment
-

Learning Outcomes

This project teaches:

- How RAG actually works
- Vector databases
- Semantic retrieval
- Context management
- AI backend architecture

- Production AI pipelines
 - Retrieval optimization
-

PROJECT 2 — AI Operations Agent

Objective

Build an AI agent capable of:

- Multi-step reasoning
- Tool usage
- Task execution
- Workflow automation

The goal is to move from:

Question-answering → Action-taking systems.

Example Capabilities

- Invoice analysis
 - Shipment generation
 - Summarization
 - Calculator tool
 - Database querying
 - Workflow execution
-

Technologies Planned

- LangGraph
 - CrewAI
 - OpenAI function calling
 - FastAPI
 - Redis memory
-

Core Concepts

- Tool calling
- Agent memory
- Planning

- Multi-step reasoning
 - Workflow orchestration
 - Guardrails
-

Planned Architecture

User ↓ Agent ↓ Planner ↓ Tool Selection ↓ Execution ↓ Final Response

Planned Features

- Agent memory
 - Task decomposition
 - Structured outputs
 - Tool execution
 - Logging
 - Evaluation
 - Guardrails
-

Learning Outcomes

This project teaches:

- Agent workflows
 - AI orchestration
 - Tool integration
 - Reasoning chains
 - Real-world AI automation
-

PROJECT 3 — Multi-Agent AI System

Objective

Build a collaborative multi-agent system where multiple specialized agents work together.

Planned Agent Roles

Planner Agent

Breaks down user tasks.

Research Agent

Handles retrieval/RAG.

Analyzer Agent

Processes information.

Execution Agent

Performs tool-based actions.

Evaluator Agent

Checks correctness and completeness.

Technologies Planned

- CrewAI
 - LangGraph
 - Vector DB
 - Redis
 - FastAPI
-

Planned Architecture

User ↓ Planner Agent ↓ Research Agent ↓ Analyzer Agent ↓ Execution Agent ↓ Evaluator Agent ↓ Final Output

Core Concepts

- Multi-agent orchestration
 - Shared memory
 - Agent communication
 - Evaluation loops
 - Role-based reasoning
 - Agent collaboration
-

Learning Outcomes

This project teaches:

- Advanced AI system design
 - Agent collaboration
 - AI orchestration frameworks
 - Reliability patterns
 - Distributed reasoning systems
-

7. Learning + Execution Strategy

Main Principle

Projects drive learning.

Courses support projects.

Not the other way around.

Daily Strategy

High-Energy Time

Focus on:

- Building
 - Debugging
 - Architecture
 - System design
-

Low-Energy Time

Focus on:

- Revision
 - Theory
 - Videos
 - Assignments
-

Weekend Strategy

Use weekends for:

- Deep work sessions
 - Major features
 - GitHub cleanup
 - Architecture improvements
-

8. Resume Positioning Strategy

The resume is being positioned toward:

- AI Engineering
- RAG Systems
- Agentic AI
- Production AI workflows
- LLMOps thinking

Key themes:

- Retrieval systems
 - Automation
 - Workflow orchestration
 - Production deployment
 - Evaluation
 - Guardrails
-

9. LinkedIn / Job Search Strategy

Target Roles

- AI Engineer
 - GenAI Engineer
 - Applied AI Engineer
 - ML Engineer
 - LLM Engineer
-

Current Job Platforms

- LinkedIn
 - WellFound
 - Naukri
 - YC Jobs
 - HuggingFace Jobs
 - Turing
 - Scaler jobs portal
-

Outreach Strategy

- Cold DMs
 - Recruiter outreach
 - Referral requests
 - GitHub showcasing
 - Portfolio-driven applications
-

10. Long-Term Vision

The long-term vision is much larger than a single project.

The goal is to eventually build:

- AI ecosystems
- Agentic platforms
- AI-native products
- Advanced LLM systems
- Multi-agent orchestration platforms

Inspired by:

- Modern GenAI systems
 - Production AI architecture
 - Foundational AI research
 - Future beyond current transformer limitations
-

11. Current Status Snapshot

Completed

- Resume transition toward AI
 - RAG project foundation
 - Open-source embeddings
 - FAISS integration
 - Retrieval pipeline
 - Neighbor-aware retrieval
 - FastAPI backend
 - Streamlit UI
 - Groq migration planning
-

In Progress

- Latency optimization
 - UX improvements
 - Evaluation system
 - Logging
-

Upcoming

- Agent system
 - Multi-agent architecture
 - Deployment
 - Advanced evaluation
-

12. Core Engineering Philosophy

The focus is not:

"Building demos"

The focus is:

"Building reliable AI systems"

Key themes:

- System thinking
 - Reliability
 - Grounding
 - Evaluation
 - Modularity
 - Scalability
 - Production mindset
-

13. Final Direction

The roadmap ultimately evolves through:

Software Engineer ↓ AI Engineer ↓ LLM Systems Engineer ↓ Agentic AI Engineer ↓ AI Systems Architect

14. Guiding Principle

Do not aim to learn AI.

Aim to build AI systems.